

Линейная регрессия и sklearn

В этой домашней работе:

- Обучим линейную регрессию для предсказания цены дома;
- Научимся работать с разными типами признаков;
- Поймем, в чем отличие между разными регуляризаторами;
- Научимся пользоваться основными инструментами в `sklearn`: моделями, трансформерами и pipeline;
- Обсудим преобразования признаков и целевой переменной, которые могут помочь в обучении линейных моделей.

Скачайте тренировочную и тестовую выборку из соревнования на kaggle: [House Prices: Advanced Regression Techniques](#). Разместите данные рядом с тетрадкой или поправьте пути при их чтении.

```
In [1]: # Чтобы не перегружать ячейки с кодом, вынесем сюда все импорты

import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import Lasso, LinearRegression, Ridge
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import KBinsDiscretizer, OneHotEncoder, PolynomialFeatures, StandardScaler

sns.set_theme(style="darkgrid")
warnings.simplefilter("ignore")
%matplotlib inline
```

Часть 0. Введение в линейные модели

Напомним, что линейная регрессия — это модель следующего вида:

$$a(x) = \langle w, x \rangle + w_0$$

где $w \in \mathbb{R}^d$, $w_0 \in \mathbb{R}$. Обучить линейную регрессию — значит найти w и w_0 .

В машинном обучении часто говорят об *обобщающей способности модели*, то есть о способности модели работать на новых, тестовых данных хорошо. Если модель будет идеально предсказывать выборку, на которой она обучалась, но при этом просто ее запомнит, не "вытащив" из данных никакой закономерности, от нее будет мало толку. Такую модель называют *переобученной*: она слишком подстроилась под обучающие примеры, не выявив никакой полезной закономерности, которая позволила бы ей совершать хорошие предсказания на данных, которые она ранее не видела.

Рассмотрим следующий пример, на котором будет хорошо видно, что значит переобучение модели. Для этого нам понадобится сгенерировать синтетические данные. Рассмотрим зависимость

$$y(x) = \cos(1.5\pi x)$$

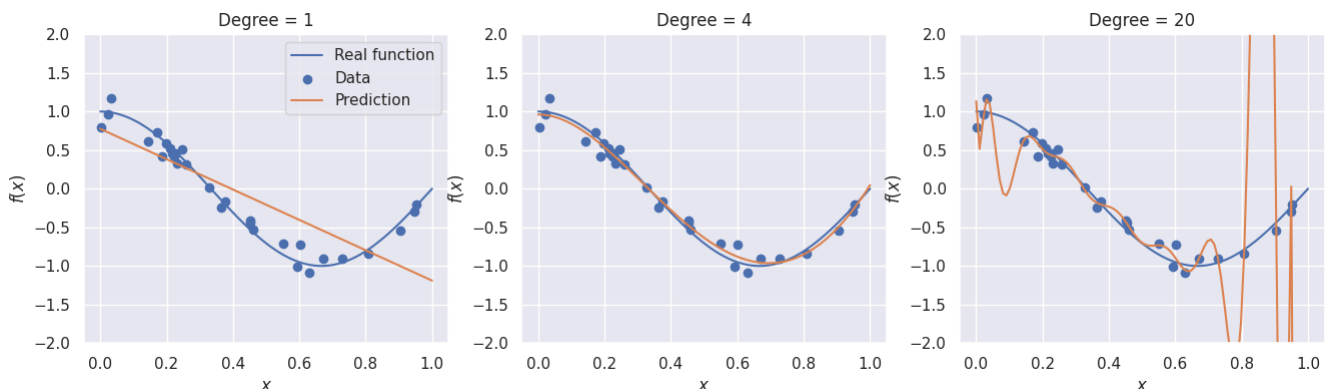
y — целевая переменная, а x — объект (просто число от 0 до 1). В жизни мы наблюдаем какое-то конечное количество пар объект-таргет, поэтому смоделируем это, взяв 30 случайных точек x_i в отрезке $[0; 1]$. Более того, в реальной жизни целевая переменная может быть зашумленной (измерения в жизни не всегда точны), смоделируем это, зашумив значение функции нормальным шумом: $\tilde{y}_i = y(x_i) + \mathcal{N}(0, 0.01)$.

Попытаемся обучить три разных линейных модели: признаки для первой — $\{x\}$, для второй — $\{x, x^2, x^3, x^4\}$, для третьей — $\{x, \dots, x^{20}\}$.

```
In [2]: np.random.seed(36)
x = np.linspace(0, 1, 100)
y = np.cos(1.5 * np.pi * x)

x_objects = np.random.uniform(0, 1, size=30)
y_objects = np.cos(1.5 * np.pi * x_objects) + np.random.normal(scale=0.1, size=x_objects.shape)

fig, axs = plt.subplots(figsize=(16, 4), ncol=3)
for i, degree in enumerate([1, 4, 20]):
    X_objects = PolynomialFeatures(degree, include_bias=False).fit_transform(x_objects[:, None])
    X = PolynomialFeatures(degree, include_bias=False).fit_transform(x[:, None])
    regr = LinearRegression().fit(X_objects, y_objects)
    y_pred = regr.predict(X)
    axs[i].plot(x, y, label="Real function")
    axs[i].scatter(x_objects, y_objects, label="Data")
    axs[i].plot(x, y_pred, label="Prediction")
    if i == 0:
        axs[i].legend()
    axs[i].set_title("Degree = %d" % degree)
    axs[i].set_xlabel("$x$")
    axs[i].set_ylabel("$f(x)$")
    axs[i].set_ylim(-2, 2)
```



Вопрос 1: Почему первая модель получилась плохой, а третья переобучилась?

Ответ: Первая модель является линейной, поэтому с помощью нее очень сложно предсказывать нелинейные зависимости. Третья же модель просто запомнила всю выборку, т.к. является многочленом слишком высокой степени

Чтобы избежать переобучения, модель регуляризуют. Обычно переобучения в линейных моделях связаны с большими весами, а поэтому модель часто штрафуют за большие значения весов, добавляя к функционалу качества, например, квадрат ℓ^2 -нормы вектора w :

$$Q_{reg}(X, y, a) = Q(X, y, a) + \lambda \|w\|_2^2$$

Это слагаемое называют ℓ_2 -регуляризатором, а коэффициент λ — коэффициентом регуляризации.

Вопрос 2: Почему большие веса в линейной модели — плохо?

Ответ: Большие веса могут привести к переобучению модели. Так же большой вес для признака, казалось бы значащий его больший вклад в предсказание модели, может быть следствием переобучения, а не действительной важности признака. При этом малейшее изменение такого признака может очень сильно повлиять на предсказание модели, если есть, например, какие-либо шумы.

Вопрос 3: Почему регуляризовать w_0 — плохая идея?

Ответ: Т.к. свободный член не влияет на зависимость модели от признаков и не влияет на переобучение модели.

Вопрос 4: На что влияет коэффициент λ ? Что будет происходить с моделью, если λ начать уменьшать? Что будет, если λ сделать слишком большим?

Ответ: Коэффициент влияет на штраф за большие веса модели. Чем больше λ , тем больше штраф.

Часть 1. Загружаем данные

In [3]:

```
train_data = pd.read_csv("train.csv")
train_data.head()
```

Out[3]:

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType	SaleCondition	SalePrice
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2008	WD	Normal	208500
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	2007	WD	Normal	181500
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	2008	WD	Normal	223500
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2006	WD	Abnorml	140000
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12	2008	WD	Normal	250000

5 rows × 81 columns

In [4]:

```
train_data.shape
```

Out[4]:

```
(1460, 81)
```

In [5]:

```
train_data.columns
```

Out[5]:

```
Index(['Id', 'MSSubClass', 'MSZoning', 'LotFrontage', 'LotArea', 'Street',
      'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig',
      'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType',
      'HouseStyle', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd',
      'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType',
      'MasVnrArea', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual',
      'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinSF1',
      'BsmtFinType2', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'Heating',
      'HeatingQC', 'CentralAir', 'Electrical', '1stFlrSF', '2ndFlrSF',
      'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath',
      'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'KitchenQual',
      'TotRmsAbvGrd', 'Functional', 'Fireplaces', 'FireplaceQu', 'GarageType',
      'GarageYrBlt', 'GarageFinish', 'GarageCars', 'GarageArea', 'GarageQual',
      'GarageCond', 'PavedDrive', 'WoodDeckSF', 'OpenPorchSF',
      'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'PoolQC',
      'Fence', 'MiscFeature', 'MiscVal', 'MoSold', 'YrSold', 'SaleType',
      'SaleCondition', 'SalePrice'],
      dtype='object')
```

Первое, что стоит заметить — у нас в данных есть уникальное для каждого объекта поле `id`. Обычно такие поля только мешают и способствуют переобучению. Удалим это поле из данных.

Выделим валидационную выборку, а также отделим значения целевой переменной от данных.

Вопрос 1: Почему поля типа `id` могут вызвать переобучение модели (не обязательно линейной)?

Ответ: эти поля не дают никакой полезной информации о природе целевой переменной. Модель может запомнить случайную связь 'id' с целевой переменной которая не отражает действительности.

Вопрос 2: Почему стоит дополнительно отделять валидационную выборку?

Ответ: для подбора гиперпараметров.

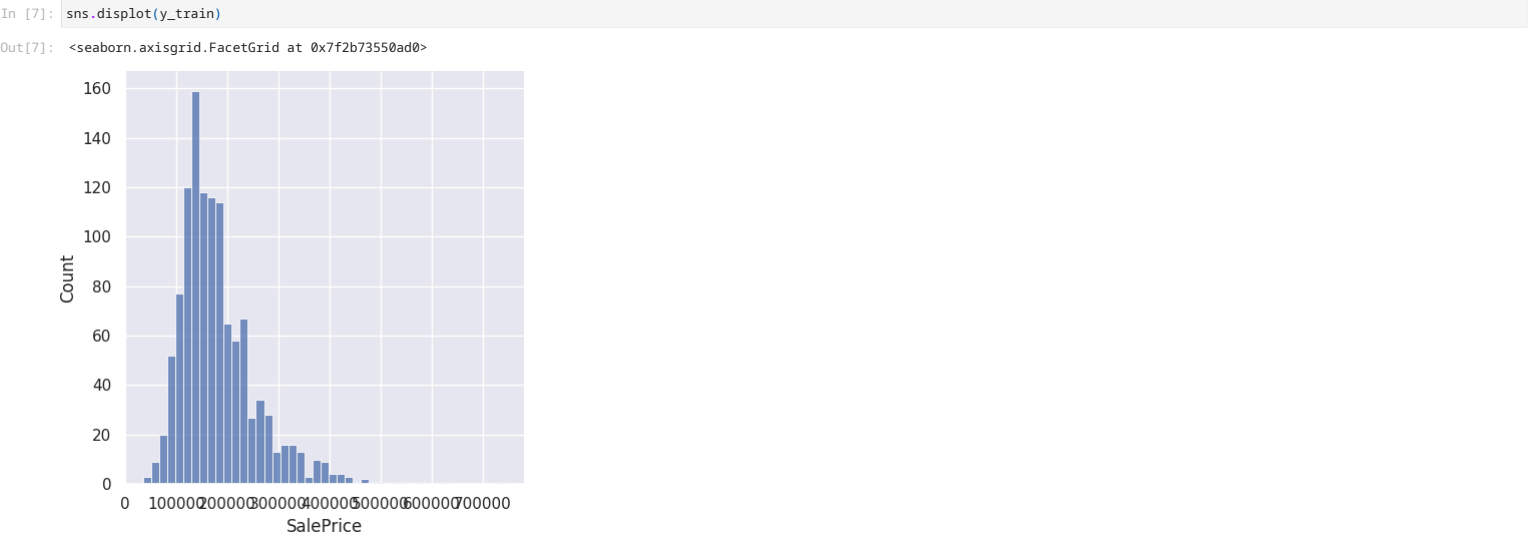
Вопрос 3: Обратите внимание на фиксацию `random_state` при сплите данных. Почему это важно?

Ответ: для того, чтобы сохранить воспроизводимость.

In [6]:

```
del train_data["Id"]
y = train_data["SalePrice"]
del train_data["SalePrice"]
X = train_data
X_train, X_val, y_train, y_val = train_test_split(X, y, train_size=0.8, random_state=42)
```

Посмотрим сначала на значения целевой переменной.



Судя по гистограмме, у нас есть примеры с нетипично большой стоимостью, что может помешать нам, если наша функция потерь слишком чувствительна к выбросам. В дальнейшем мы рассмотрим способы, как минимизировать ущерб от этого.

Так как для решения нашей задачи мы бы хотели обучить линейную регрессию, было бы хорошо найти признаки, "наиболее линейно" связанные с целевой переменной, иначе говоря, посмотреть на коэффициент корреляции Пирсона между признаками и целевой переменной. Заметим, что не все признаки являются числовыми, пока что мы не будем рассматривать такие признаки.

Вопрос: Что означает, что коэффициент корреляции Пирсона между двумя случайными величинами равен 1? -1? 0?

Ответ: Коэффициент корреляции Пирсона равный 0 означает, что признаки не коррелируют. Если он равен -1 или 1, то это означает что признаки зависят линейно.

In [8]:

```
numeric_data = X_train.select_dtypes([np.number])
numeric_features = numeric_data.columns
numeric_data.head()
```

Out[8]:	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	BsmtFinSF2	...	GarageArea	WoodDeckSF	OpenPorchSF	EnclosedPorch	3SsnPorch	ScreenPorch
	254	20	70.0	8400	5	6	1957	1957	0.0	922	0 ...	294	250	0	0	0	
	1066	60	59.0	7837	6	7	1993	1994	0.0	0	0 ...	380	0	40	0	0	
	638	30	67.0	8777	5	7	1910	1950	0.0	0	0 ...	0	328	0	164	0	
	799	50	60.0	7200	5	7	1937	1950	252.0	569	0 ...	240	0	0	264	0	
	380	50	50.0	5000	5	6	1924	1950	0.0	218	0 ...	308	0	0	242	0	

5 rows × 36 columns



Заметим, что в данных присутствуют пропуски, заполним их средним значением по признаку.

Вопрос: Как правильно заполнять пропуски для валидационной и тестовой выборки?

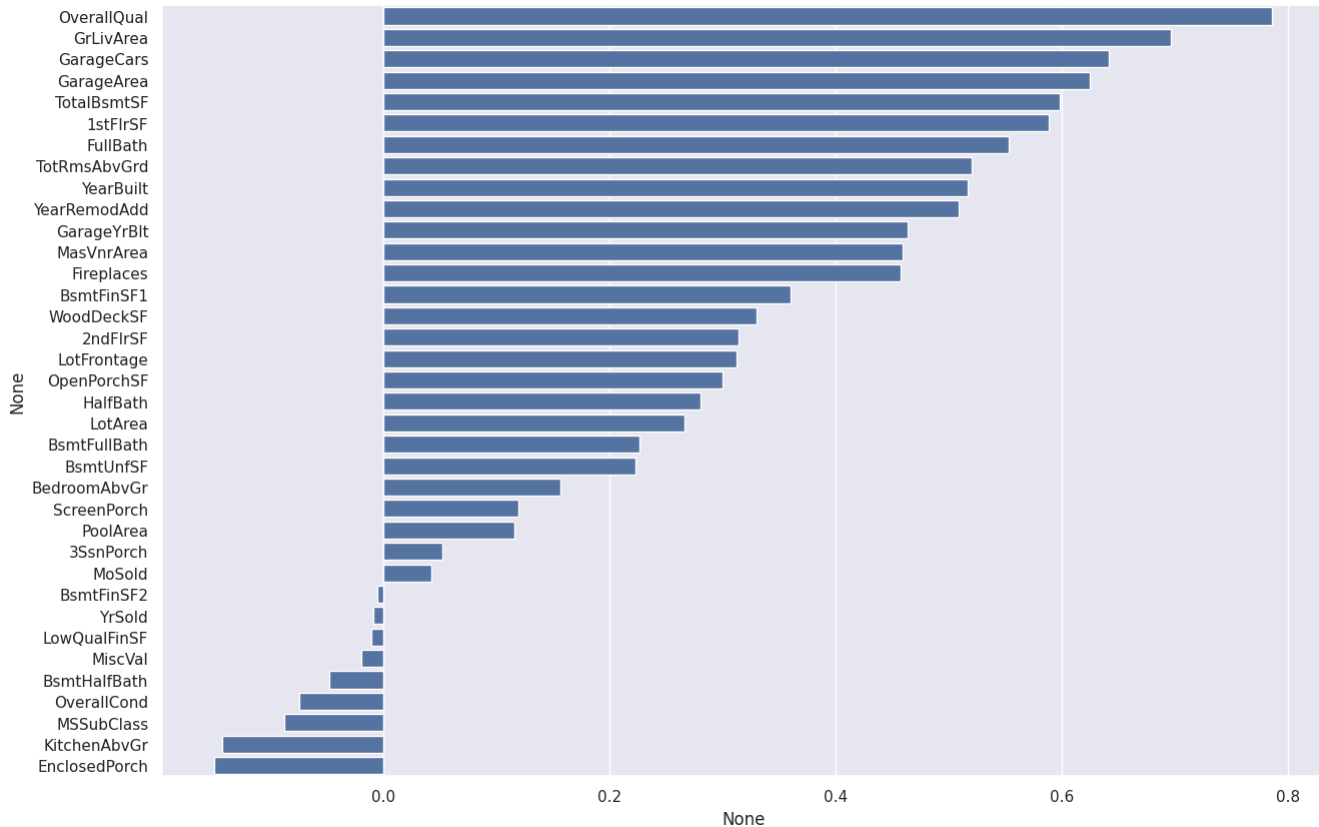
Ответ: Лучше заполнять средними значениями отдельно, чтобы информация о валидационной выборке не появилась в тренировочной.

```
In [9]: for col in numeric_features:
X_train_mean = X_train[col].mean()
X_train[col].fillna(X_train_mean, inplace=True)

X_val_mean = X_val[col].mean()
X_val[col].fillna(X_val_mean, inplace=True)
```

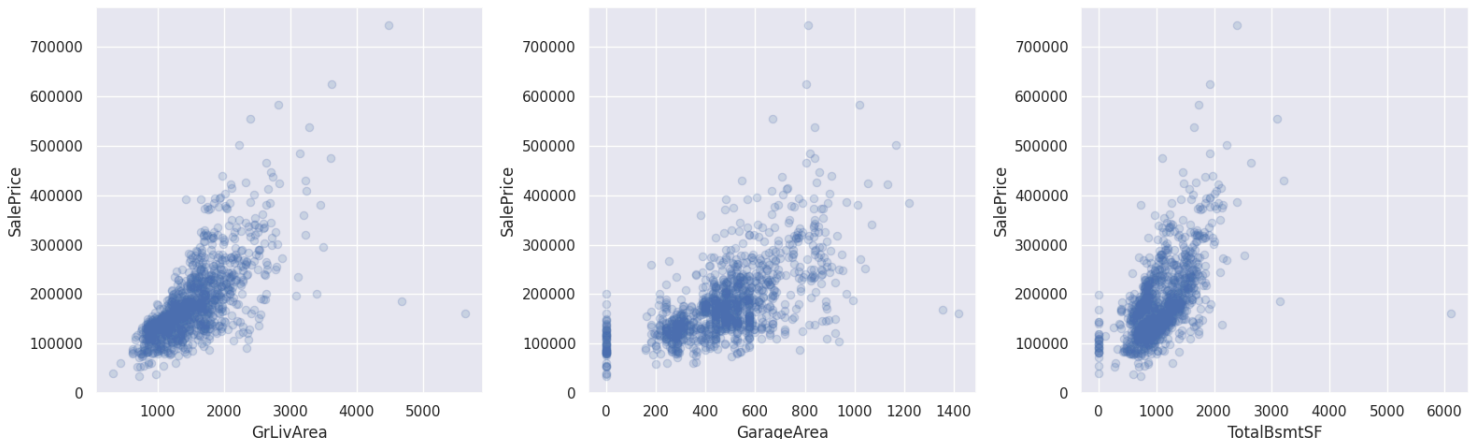
```
In [10]: correlations = X_train[numeric_features].corrwith(y_train).sort_values(ascending=False)

plot = sns.barplot(y=correlations.index, x=correlations)
plot.figure.set_size_inches(15, 10)
```



Посмотрим на признаки из начала списка. Для этого нарисует график зависимости целевой переменной от каждого из признаков. На этом графике каждая точка соответствует паре признак-таргет (такие графики называются scatter-plot).

```
In [11]: fig, axs = plt.subplots(figsize=(16, 5), ncols=3)
for i, feature in enumerate(["GrLivArea", "GarageArea", "TotalBsmtSF"]):
axs[i].scatter(X_train[feature], y_train, alpha=0.2)
axs[i].set_xlabel(feature)
axs[i].set_ylabel("SalePrice")
plt.tight_layout()
```



Видим, что между этими признаками и целевой переменной действительно наблюдается линейная зависимость.

Часть 2. Первая модель

Немного об обучении моделей. В арсенале ML-специалиста кроме `pandas` и `matplotlib` должны быть библиотеки, позволяющие обучать модели. Для простых моделей (линейные модели, решающее дерево, ...) отлично подходит `sklearn`: в нем очень понятный и простой интерфейс. Несмотря на то, что в `sklearn` есть реализация бустинга и простых нейронных сетей, ими все же не пользуются и предпочитают специализированные библиотеки: `XGBoost`, `LightGBM` и пр. для градиентного бустинга над деревьями, `PyTorch`, и пр. для нейронных сетей. Так как мы будем обучать линейную регрессию, нам подойдет реализация из `sklearn`.

Попробуем обучить линейную регрессию на числовых признаках из нашего датасета. В `sklearn` есть несколько классов, реализующих линейную регрессию:

- `LinearRegression` — "классическая" линейная регрессия с оптимизацией MSE. Веса находятся как точное решение: $w^* = (X^T X)^{-1} X^T y$
- `Ridge` — линейная регрессия с оптимизацией MSE и ℓ_2 -регуляризацией
- `Lasso` — линейная регрессия с оптимизацией MSE и ℓ_1 -регуляризацией

У моделей из `sklearn` есть методы `fit` и `predict`. Первый принимает на вход обучающую выборку и вектор целевых переменных и обучает модель, второй, будучи вызванным после обучения модели, возвращает предсказание на выборке. Попробуем обучить нашу первую модель на числовых признаках, которые у нас сейчас есть:

```
In [12]: model = Ridge()
model.fit(X_train[numeric_features], y_train)
y_pred = model.predict(X_val[numeric_features])
y_train_pred = model.predict(X_train[numeric_features])
```

Стандартный способ оценить качество регрессии — **MSE**

```
In [13]: def mean_squared_error(y_true: np.ndarray, y_pred: np.ndarray, squared: bool = True) -> float:
        """Calculate Mean Squared Error

        Args:
            y_true: array with ground truth values, [n_samples,]
            y_pred: array with predicted values, [n_samples,]
            squared: whether to return squared MSE or not

        Returns:
            number, calculated error
        """

        mse = sum((y1 - y2)**2 for y1, y2 in zip(y_true, y_pred)) / len(y_true)
        rmse = np.sqrt(mse)

        if squared:
            return mse
        else:
            return rmse
```

```
In [14]: print("Val RMSE = %.4f" % mean_squared_error(y_val, y_pred, squared=False))
print("Train RMSE = %.4f" % mean_squared_error(y_train, y_train_pred, squared=False))

Val RMSE = 36832.8157
Train RMSE = 33925.4760
```

Мы обучили первую модель и даже посчитали ее качество на отложенной выборке! Давайте теперь посмотрим на то, как можно оценить качество модели с помощью кросс-валидации. Принцип кросс-валидации изображен на рисунке



При кросс-валидации мы делим обучающую выборку на n частей (fold). Затем мы обучаем n моделей: каждая модель обучается при отсутствии соответствующего фолда, то есть i -ая модель обучается на всей обучающей выборке, кроме объектов, которые попали в i -ый фолд (out-of-fold). Затем мы измеряем качество i -ой модели на i -ом фолде. Так как он не участвовал в обучении этой модели, мы получим "честный результат". После этого, для получения финального значения метрики качества, мы можем усреднить полученные нами n значений.

```
In [15]: cv_scores = cross_val_score(model, X_train[numeric_features], y_train, cv=10, scoring="neg_root_mean_squared_error")
print("Cross validation scores:\n\t", "\n\t".join("%.4f" % x for x in cv_scores))
print("Mean CV MSE = %.4f" % np.mean(-cv_scores))

Cross validation scores:
-27391.0974
-47008.8112
-28698.2996
-45751.1960
-67322.8753
-38519.7913
-27438.8056
-23716.7933
-26690.8362
-27455.2285

Mean CV MSE = 35999.3734
```

Обратите внимание на то, что результаты `cv_scores` получились отрицательными. Это соглашение в `sklearn` (скоринговую функцию нужно максимизировать). Поэтому все стандартные скореры называются `neg_*`, например, `neg_root_mean_squared_error`.

Обратите внимание, что по отложенной выборке и при кросс-валидации мы считаем RMSE (Root Mean Squared Error), хотя в функционале ошибки при обучении модели используется MSE.

$$\text{RMSE}(X, y, a) = \sqrt{\frac{1}{\ell} \sum_{i=1}^{\ell} (y_i - a(x_i))^2}$$

Вопрос: Почему оптимизация RMSE эквивалентна оптимизации MSE?

Ответ: Т.к. обе функции являются одинаково монотонными, и если минимизировать MSE, то автоматически минимизируется и RMSE.

Для того, чтобы иметь некоторую точку отсчета, удобно посчитать оптимальное значение функции потерь при константном предсказании.

Вопрос: Чему равна оптимальная константа для RMSE?

Ответ: среднему арифметическому

```
In [16]: best_constant = y_val.mean()
```

```
In [17]: print(
    "Test RMSE with best constant = %.4f"
    % mean_squared_error(y_val, best_constant * np.ones(y_val.shape), squared=False)
)
print(
    "Train RMSE with best constant = %.4f"
    % mean_squared_error(y_train, best_constant * np.ones(y_train.shape), squared=False)
)
```

Test RMSE with best constant = 87580.3985
Train RMSE with best constant = 77274.3126

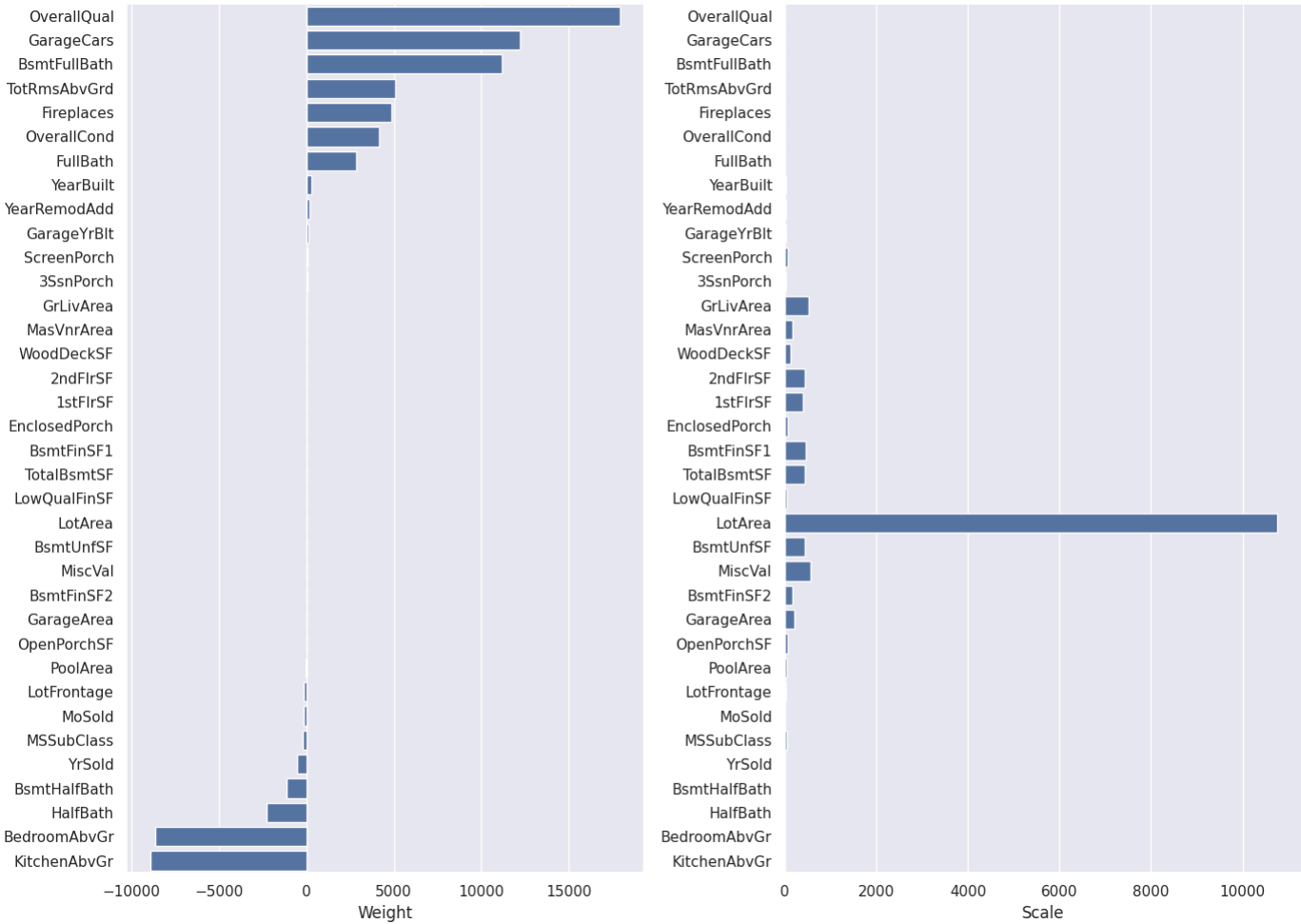
Давайте посмотрим на то, какие же признаки оказались самыми "сильными". Для этого визуализируем веса, соответствующие признакам. Чем больше вес — тем более сильным является признак.

Вопрос: Почему это не совсем правда?

Ответ: признаки не отмасштабированы.

```
In [18]: def show_weights(features, weights, scales):
fig, axs = plt.subplots(figsize=(14, 10), ncols=2)
sorted_weights = sorted(zip(weights, features, scales), reverse=True)
weights = [x[0] for x in sorted_weights]
features = [x[1] for x in sorted_weights]
scales = [x[2] for x in sorted_weights]
sns.barplot(y=weights, x=weights, ax=axs[0])
axs[0].set_xlabel("Weight")
sns.barplot(y=features, x=scales, ax=axs[1])
axs[1].set_xlabel("Scale")
plt.tight_layout()
```

```
In [19]: show_weights(numeric_features, model.coef_, X_train[numeric_features].std())
```



Будем масштабировать наши признаки перед обучением модели. Это, среди, прочего, сделает нашу регуляризацию более честной: теперь все признаки будут регуляризоваться в равной степени.

Для этого воспользуемся трансформером `StandardScaler`. Трансформеры в `sklearn` имеют методы `fit` и `transform` (а еще `fit_transform`). Метод `fit` принимает на вход обучающую выборку и считает по ней необходимые значения (например статистики, как `StandardScaler`: среднее и стандартное отклонение каждого из признаков). `transform` применяет преобразование к переданной выборке.

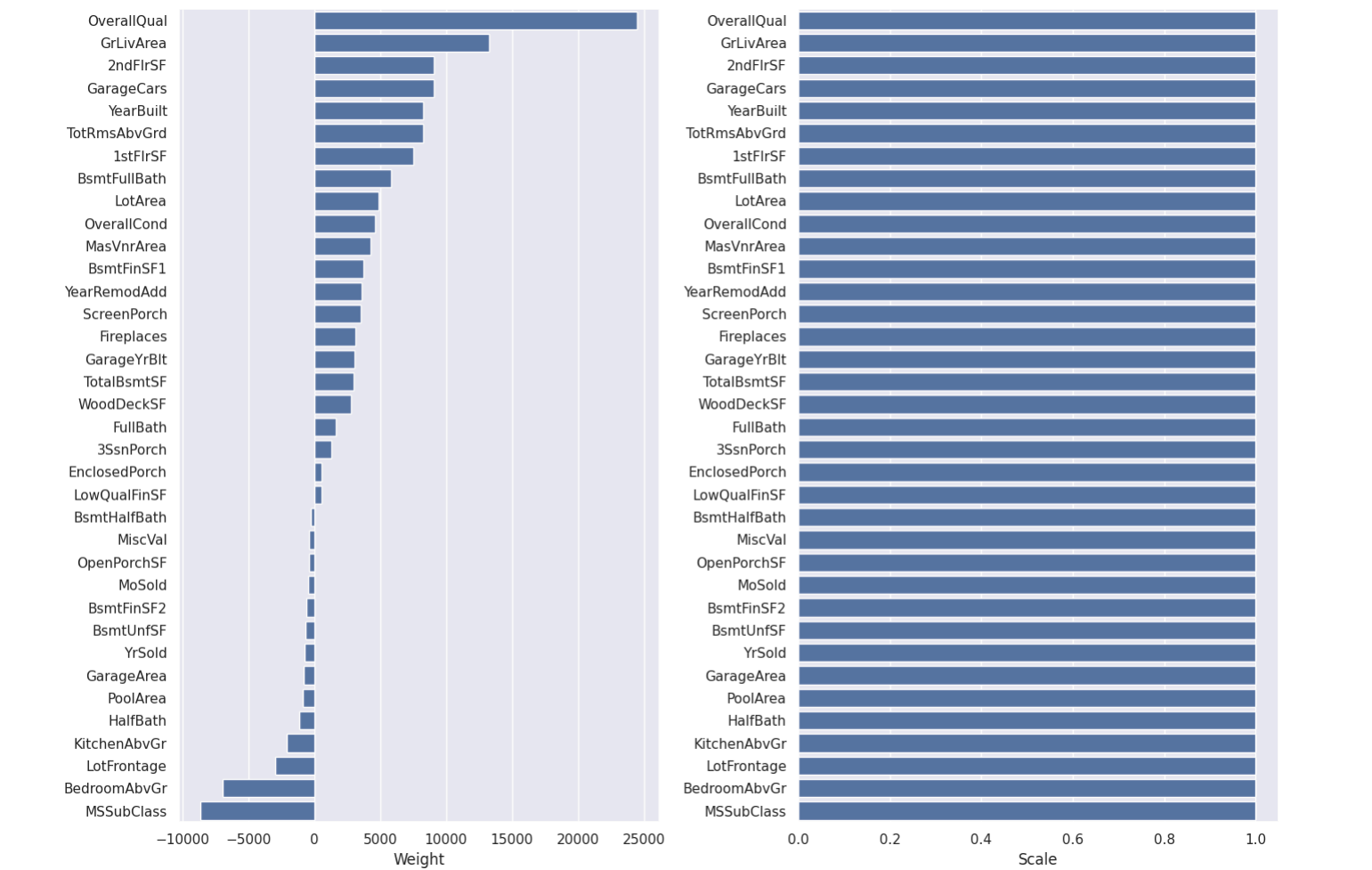
```
In [20]: X_train_scaled = StandardScaler().fit_transform(X_train[numeric_features])
X_val_scaled = StandardScaler().fit_transform(X_val[numeric_features])
```

```
In [21]: model = Ridge()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_val_scaled)
y_train_pred = model.predict(X_train_scaled)

print("Test RMSE = %.4f" % mean_squared_error(y_val, y_pred, squared=False))
print("Train RMSE = %.4f" % mean_squared_error(y_train, y_train_pred, squared=False))

Test RMSE = 36827.7732
Train RMSE = 33925.4481
```

```
In [22]: scales = pd.Series(data=X_train_scaled.std(axis=0), index=numeric_features)
show_weights(numeric_features, model.coef_, scales)
```



Наряду с параметрами (веса w , w_0), которые модель оптимизирует на этапе обучения, у модели есть и гиперпараметры. У нашей модели это `alpha` — коэффициент регуляризации. Подбирают его обычно по сетке, измеряя качество на валидационной (не тестовой) выборке или с помощью кросс-валидации. Посмотрим, как это можно сделать.

Для начала зададим возможные значения гиперпараметра, воспользуемся `np.logspace`, чтобы узнать оптимальный порядок величины. Ограничим допустимые значения 10^{-2} и 10^3 , возьмем 20 точек.

```
In [23]: alphas = np.logspace(-2, 3, num=20)
assert alphas[0] == 1e-2
assert alphas[-1] == 1e3
assert len(alphas) == 20

alphas

Out[23]: array([1.00000000e-02, 1.83298071e-02, 3.35981829e-02, 6.15848211e-02,
1.12883789e-01, 2.06913808e-01, 3.79269019e-01, 6.95192796e-01,
1.27427499e+00, 2.33572147e+00, 4.28133240e+00, 7.84759970e+00,
1.43844989e+01, 2.63665090e+01, 4.83293024e+01, 8.85866790e+01,
1.62377674e+02, 2.97635144e+02, 5.45559478e+02, 1.00000000e+03])

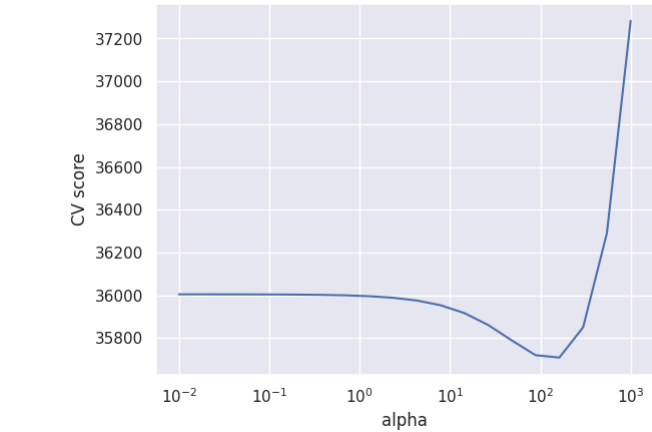
In [24]: searcher = GridSearchCV(Ridge(), [{"alpha": alphas}], scoring="neg_root_mean_squared_error", cv=10)
searcher.fit(X_train_scaled, y_train)

best_alpha = searcher.best_params_["alpha"]
print("Best alpha = %.4f" % best_alpha)

plt.plot(alphas, -searcher.cv_results_["mean_test_score"])
plt.xscale("log")
plt.xlabel("alpha")
plt.ylabel("CV score")

Best alpha = 162.3777

Out[24]: Text(0, 0.5, 'CV score')
```



Вопрос: Почему мы не подбираем коэффициент регуляризации по обучающей выборке? По тестовой выборке?

Ответ: коэффициент регуляризации является гиперпараметром, а значит для избежания переобучения необходимо подбирать его исключительно на валидационной выборке.

Попробуем обучить модель с подобранным коэффициентом регуляризации. Заодно воспользуемся очень удобным классом `Pipeline`: обучение модели часто представляется как последовательность некоторых действий с обучающей и тестовой выборками (например, сначала нужно отмасштабировать выборку (причем для обучающей выборки нужно применить метод `fit`, а для тестовой — `transform`), а затем обучить/применить модель (для обучающей `fit`, а для тестовой — `predict`). `Pipeline` позволяет хранить эту последовательность шагов и корректно обрабатывает разные типы выборок: и обучающую, и тестовую.

```
In [25]: simple_pipeline = Pipeline([("scaling", StandardScaler()), ("regression", Ridge(best_alpha))])

model = simple_pipeline.fit(X_train[numeric_features], y_train)
y_pred = model.predict(X_val[numeric_features])
print("Test RMSE = %.4f" % mean_squared_error(y_val, y_pred, squared=False))

Test RMSE = 37147.8534
```

Часть 3. Работаем с категориальными признаками

Сейчас мы явно вытягиваем из данных не всю информацию, что у нас есть, просто потому, что мы не используем часть признаков. Эти признаки в датасете закодированы строками, каждый из них обозначает некоторую категорию. Такие признаки называются категориальными. Давайте выделим такие признаки и сразу заполним пропуски в них специальным значением (то, что у признака пропущено значение, само по себе может быть хорошим признаком).

In [26]:

categorical = list(X_train.dtypes[X_train.dtypes == "object"].index)
X_train[categorical].head()

Out [26]:

	MSZoning	Street	Alley	LotShape	LandContour	Utilities	LotConfig	LandSlope	Neighborhood	Condition1	...	GarageType	GarageFinish	GarageQual	GarageCond	PavedDrive	PoolQC	Fence	MiscFeatu
254	RL	Pave	NaN	Reg	Lvl	AllPub	Inside	Gtl	NAmes	Norm	...	Attchd	RFn	TA	TA	Y	NaN	NaN	NaN
1066	RL	Pave	NaN	IR1	Lvl	AllPub	Inside	Gtl	Gilbert	Norm	...	Attchd	RFn	TA	TA	Y	NaN	NaN	NaN
638	RL	Pave	NaN	Reg	Lvl	AllPub	Inside	Gtl	Edwards	Feedr	...	NaN	NaN	NaN	NaN	P	NaN	MnPrv	NaN
799	RL	Pave	NaN	Reg	Lvl	AllPub	Corner	Gtl	SWISU	Feedr	...	Detchd	Unf	TA	TA	Y	NaN	MnPrv	NaN
380	RL	Pave	Pave	Reg	Lvl	AllPub	Inside	Gtl	SWISU	Norm	...	Detchd	Unf	TA	TA	Y	NaN	NaN	NaN

5 rows × 43 columns

В категориальных данных также есть пропуски, заполним их отдельной новой категорией `NotGiven`

In [27]:

for col in categorical:
 X_train[col].fillna("NotGiven", inplace=True)
 X_val[col].fillna("NotGiven", inplace=True)

In [28]:

X_train[categorical].sample(5)

Out [28]:

	MSZoning	Street	Alley	LotShape	LandContour	Utilities	LotConfig	LandSlope	Neighborhood	Condition1	...	GarageType	GarageFinish	GarageQual	GarageCond	PavedDrive	PoolQC	Fence	MiscFeatu
384	RL	Pave	NotGiven	IR2	Low	AllPub	Corner	Mod	ClearCr	Feedr	...	Attchd	Fin	TA	TA	Y	NotGiven	NotGiven	NaN
977	FV	Pave	Pave	IR1	Lvl	AllPub	Inside	Gtl	Somerst	Norm	...	Attchd	Fin	TA	TA	Y	NotGiven	NotGiven	NaN
1097	RL	Pave	NotGiven	Reg	Lvl	AllPub	Inside	Gtl	StoneBr	Norm	...	Attchd	RFn	TA	TA	Y	NotGiven	NotGiven	NaN
1140	RL	Pave	NotGiven	Reg	Lvl	AllPub	Corner	Gtl	NAmes	Norm	...	Detchd	Unf	TA	TA	Y	NotGiven	NotGiven	NaN
918	RL	Pave	NotGiven	IR1	Lvl	AllPub	Corner	Gtl	SawyerW	Norm	...	Attchd	RFn	TA	TA	Y	NotGiven	GdPrv	NaN

5 rows × 43 columns

Сейчас нам нужно как-то закодировать эти категориальные признаки числами, ведь линейная модель не может работать с такими абстракциями. Два стандартных трансформера из `sklearn` для работы с категориальными признаками — `OrdinalEncoder` (просто перенумеровывает значения признака натуральными числами) и `OneHotEncoder`.

`OneHotEncoder` ставит в соответствие каждому признаку целый вектор, состоящий из нулей и одной единицы (которая стоит на месте, соответствующем принимаемому значению, таким образом кодируя его).

Вопрос: Проинтерпретируйте, что означают веса модели перед `OneHot`-кодированными признаками. Почему пользоваться `OrdinalEncoder` в случае линейной модели — скорее плохой вариант? Какие недостатки есть у `OneHot`-кодирования?

Ответ: Веса перед `OneHot`-кодированными признаками свидетельствуют о наличии или отсутствии того или иного признака. `OrdinalEncoder` -- плохой вариант, т.к. может создать иерархию внутри признаков даже если ее нет. Основным недостатком `OneHot` кодирования является увеличение размерности пространства признаков

In [29]:

column_transformer = ColumnTransformer(
 [("ohe", OneHotEncoder(handle_unknown="ignore"), categorical)], ("scaling", StandardScaler(), numeric_features))
)

pipeline = Pipeline(steps=[("ohe_and_scaling", column_transformer), ("regression", Ridge())])

model = pipeline.fit(X_train, y_train)
y_pred = model.predict(X_val)
print("Test RMSE = %.4f" % mean_squared_error(y_val, y_pred, squared=False))

Test RMSE = 29744.0343

Вопрос: Как вы думаете, почему мы не производим скейлинг `OneHot`-кодированных признаков?

Ответ: Это не нужно, т.к. `OneHot`-кодированные признаки имеют значения 0 или 1. Таким образом может потеряться бинарность признаков.

Посмотрим на размеры матрицы после `OneHot`-кодирования:

In [30]:

print("Size before OneHot:", X_train.shape)
print("Size after OneHot:", column_transformer.transform(X_train).shape)

Size before OneHot: (1168, 79)
Size after OneHot: (1168, 301)

Как видим, количество признаков увеличилось более, чем в 3 раза. Это может повысить риски переобучиться: соотношение количества объектов к количеству признаков сильно сократилось.

Попытаемся обучить линейную регрессию с ℓ_1 -регуляризатором.

Вопрос: Каким полезным свойством обладает такой регуляризатор?

Ответ: Может занулить некоторые веса, а значит модель отберет наиболее важные признаки таким образом уменьшив размерность.

In [31]:

column_transformer = ColumnTransformer(
 [("ohe", OneHotEncoder(handle_unknown="ignore"), categorical)], ("scaling", StandardScaler(), numeric_features))
)

lasso_pipeline = Pipeline(steps=[("ohe_and_scaling", column_transformer), ("regression", Lasso())])

model = lasso_pipeline.fit(X_train, y_train)
y_pred = model.predict(X_val)
print("RMSE = %.4f" % mean_squared_error(y_val, y_pred, squared=False))

RMSE = 29550.1432

In [32]:

ridge_zeros = np.sum(pipeline.steps[-1][-1].coef_ == 0)
lasso_zeros = np.sum(lasso_pipeline.steps[-1][-1].coef_ == 0)
print("Zero weights in Ridge:", ridge_zeros)
print("Zero weights in Lasso:", lasso_zeros)

Zero weights in Ridge: 0
Zero weights in Lasso: 35

Подберем для нашей модели оптимальный коэффициент регуляризации. Обратите внимание, как перебираются параметры у `Pipeline`.

```
In [33]: alphas = np.logspace(-2, 4, 20)
searcher = GridSearchCV(
```

```
)
    lasso_pipeline, [{"regression__alpha": alphas}], scoring="neg_root_mean_squared_error", cv=10, n_jobs=-1
)
searcher.fit(X_train, y_train)

best_alpha = searcher.best_params_["regression__alpha"]
print("Best alpha = %.4f" % best_alpha)

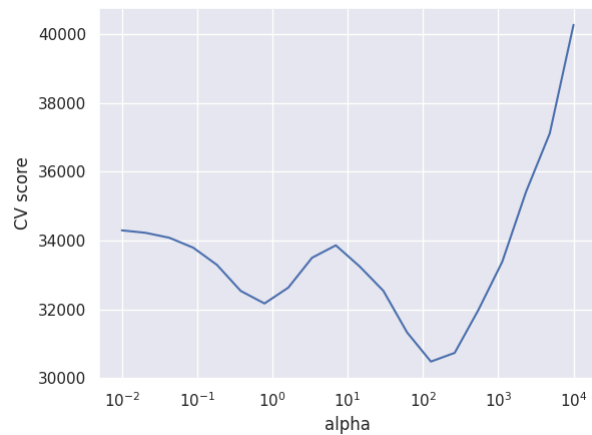
plt.plot(alphas, -searcher.cv_results_["mean_test_score"])
plt.xscale("log")
plt.xlabel("alpha")
plt.ylabel("CV score")
```

[illegible]

[illegible]

```
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 11859905662.409363, tolerance: 633179858.6205381
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 9527443782.840576, tolerance: 629439141.0479807
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 2265378097.0455933, tolerance: 640937177.9049449
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 2004829769.8796382, tolerance: 633555450.0449674
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 2010512211.7973022, tolerance: 633179858.6205381
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 1785571365.0217285, tolerance: 619229931.187113
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 1732543203.9718018, tolerance: 635058824.3188514
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 1351930141.0663452, tolerance: 603073775.0431188
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 3213840177.9729004, tolerance: 640937177.9049449
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 2141725740.2277832, tolerance: 629439141.0479807
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 2872875406.8445435, tolerance: 633555450.0449674
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 1969769618.126709, tolerance: 603073775.0431188
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 2484917212.2492676, tolerance: 619229931.187113
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 2562644720.946167, tolerance: 635058824.3188514
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 3039939612.6870117, tolerance: 629439141.0479807
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 4642613855.915527, tolerance: 640937177.9049449
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 2879485597.866333, tolerance: 633179858.6205381
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 3005990101.677185, tolerance: 603073775.0431188
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 4405208142.87793, tolerance: 629439141.0479807
model = cd_fast.sparse_enet_coordinate_descent(
/home/iraedeus/Projects/spbu_sem4_DT/.venv/lib64/python3.13/site-packages/sklearn/linear_model/_coordinate_descent.py:656: ConvergenceWarning: Objective did not converge. You might want to
increase the number of iterations. Duality gap: 3994049530.9364014, tolerance: 635058824.3188514
model = cd_fast.sparse_enet_coordinate_descent(
Best alpha = 127.4275
```

```
Out[33]: Text(0, 0.5, 'CV score')
```



```
In [34]: column_transformer = ColumnTransformer(
        [ ("ohe", OneHotEncoder(handle_unknown="ignore"), categorical), ("scaling", StandardScaler(), numeric_features)]
    )

    pipeline = Pipeline(steps=[ ("ohe_and_scaling", column_transformer), ("regression", Lasso(best_alpha))])

    model = pipeline.fit(X_train, y_train)
    y_pred = model.predict(X_val)
    print("Test RMSE = %.4f" % mean_squared_error(y_val, y_pred, squared=False))

Test RMSE = 28700.2794
```

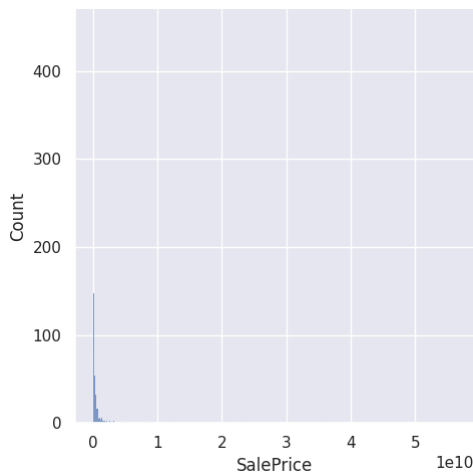
```
In [35]: lasso_zeros = np.sum(pipeline.steps[-1][-1].coef_ == 0)
        print("Zero weights in Lasso:", lasso_zeros)
```

Zero weights in Lasso: 195

Иногда очень полезно посмотреть на распределение остатков. Нарисуем гистограмму распределения квадратичной ошибки на обучающих объектах:

```
In [36]: error = (y_train - model.predict(X_train)) ** 2
        sns.displot(error)
```

```
Out[36]: <seaborn.axisgrid.FacetGrid at 0x7f2b6c82c410>
```



Как видно из гистограммы, есть примеры с очень большими остатками. Попробуем их выбросить из обучающей выборки. Например, выбросим примеры, остаток у которых больше 0.95-квантили.

```
In [37]: mask = error < np.quantile(error, 0.95)

In [38]: column_transformer = ColumnTransformer(
    [ ("ohe", OneHotEncoder(handle_unknown="ignore"), categorical), ("scaling", StandardScaler(), numeric_features)]
)

pipeline = Pipeline(steps=[("ohe_and_scaling", column_transformer), ("regression", Lasso(best_alpha))])

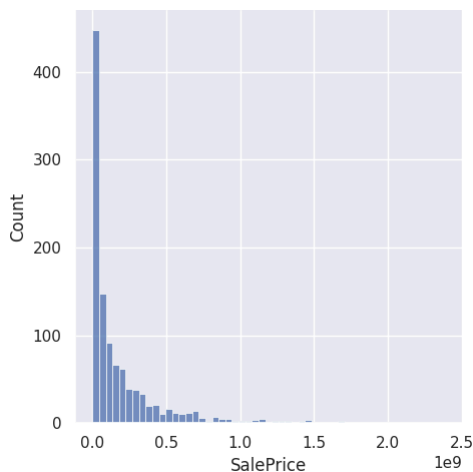
model = pipeline.fit(X_train[mask], y_train[mask])
y_pred = model.predict(X_val)
print("Test RMSE = %.4f" % mean_squared_error(y_val, y_pred, squared=False))

Test RMSE = 28502.1736
```

```
In [39]: X_train = X_train[mask]
y_train = y_train[mask]
```

```
In [40]: error = (y_train[mask] - model.predict(X_train[mask])) ** 2
sns.displot(error)
```

Out[40]: <seaborn.axisgrid.FacetGrid at 0x7f2b6c015d10>



Видим, что качество модели заметно улучшилось! Также бывает очень полезно посмотреть на примеры с большими остатками и попытаться понять, почему же модель на них так сильно ошибается: это может дать понимание, как модель можно улучшить.

Часть 4. Подготовка данных для линейных моделей

Есть важное понятие, связанное с применением линейных моделей, — *спрямляющее пространство*. Под ним понимается такое признаковое пространство для наших объектов, в котором линейная модель хорошо описывает данные, даёт хорошее качество прогнозов.

Не существует общих рекомендаций о том, как найти спрямляющее пространство для произвольной выборки. Есть лишь некоторые общие советы — например, если добавить в выборку полиномиальных признаков, то скорее всего модель станет работать лучше (если не переобучится). Есть и другие трюки.

У линейных моделей есть огромное преимущество: они имеют мало параметров, а поэтому их можно обучить даже на небольшой выборке. Если выборка большая, то параметры модели получится оценить более надёжно — но в то же время есть риск, что данные будут слишком разнообразными, чтобы линейная модель могла уловить все закономерности в них. Иногда можно улучшить ситуацию путём разбиения признакового пространства на несколько областей и построения своей модели в каждой из них.

Попробуем для примера в нашей задаче разделить выборку на две части по признаку OverallQual. Это один из самых сильных признаков, и, возможно, разбиение по нему даст нам две выборки с заведомо разными ценами на дома.

Для начала вспомним, какое качество получается у обычной гребневой регрессии.

```
In [41]: column_transformer = ColumnTransformer(
    [ ("ohe", OneHotEncoder(handle_unknown="ignore"), categorical), ("scaling", StandardScaler(), numeric_features)]
)

pipeline = Pipeline(steps=[("ohe_and_scaling", column_transformer), ("regression", Ridge())])

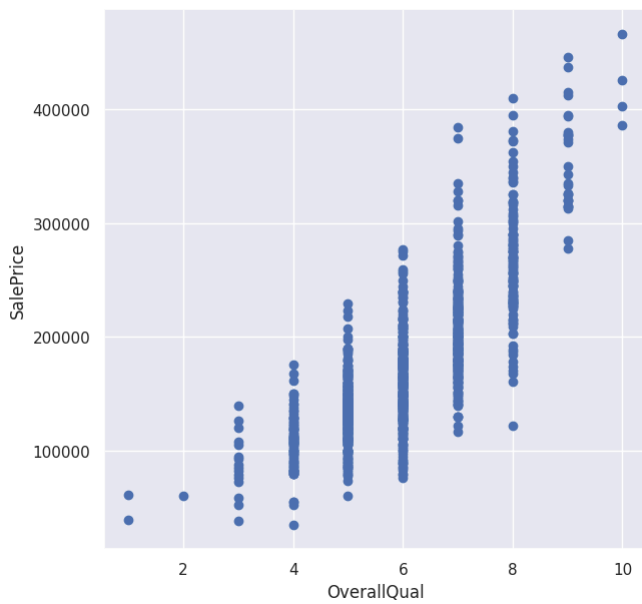
model = pipeline.fit(X_train, y_train)
y_pred = model.predict(X_val)
print("Test RMSE = %.4f" % mean_squared_error(y_val, y_pred, squared=False))

Test RMSE = 28153.4497

Посмотрим на связь OverallQual и целевой переменной.
```

```
In [42]: plt.figure(figsize=(7, 7))
plt.scatter(X_train["OverallQual"], y_train)
plt.xlabel("OverallQual")
plt.ylabel("SalePrice")
```

Out[42]: Text(0, 0.5, 'SalePrice')



```
In [43]: threshold = 5
mask = X_train["OverallQual"] <= threshold
X_train_1 = X_train[mask]
y_train_1 = y_train[mask]
X_train_2 = X_train[~mask]
y_train_2 = y_train[~mask]
```

```
In [44]: column_transformer1 = ColumnTransformer(
    [ ("ohe", OneHotEncoder(handle_unknown="ignore"), categorical), ("scaling", StandardScaler(), numeric_features)]
)

pipeline1 = Pipeline(steps=[("ohe_and_scaling", column_transformer1), ("regression", Ridge())])

column_transformer2 = ColumnTransformer(
    [ ("ohe", OneHotEncoder(handle_unknown="ignore"), categorical), ("scaling", StandardScaler(), numeric_features)]
)

pipeline2 = Pipeline(steps=[("ohe_and_scaling", column_transformer2), ("regression", Ridge())])

model1 = pipeline1.fit(X_train_1, y_train_1)
model2 = pipeline2.fit(X_train_2, y_train_2)

y_pred_1 = model1.predict(X_val)
y_pred_2 = model2.predict(X_val)
mask_test = X_val["OverallQual"] <= threshold
y_pred = y_pred_1.copy()
y_pred[~mask_test] = y_pred_2[~mask_test]

print("Test RMSE = %.4f" % mean_squared_error(y_val, y_pred, squared=False))
```

Test RMSE = 27242.3091

Получилось лучше! И это при практически случайном выборе разбиения. Если бы мы поработали над этим получше, то и качество, скорее всего, получилось бы выше.

Перейдём к следующему трюку — бинаризации признаков. Мы выбираем n порогов $t_1, \dots, t_n$ для признака x_j и генерируем $n + 1$ новый признак: $[x_j \leq t_1], [t_1 < x_j \leq t_2], \dots, [t_{n-1} < x_j \leq t_n], [x_j > t_n]$. Такое преобразование может неплохо помочь в случае, если целевая переменная нелинейно зависит от одного из признаков. Рассмотрим синтетический пример.

```
In [45]: x_plot = np.linspace(0, 1, 10000)

X = np.random.uniform(0, 1, size=30)
y = np.cos(1.5 * np.pi * X) + np.random.normal(scale=0.1, size=X.shape)

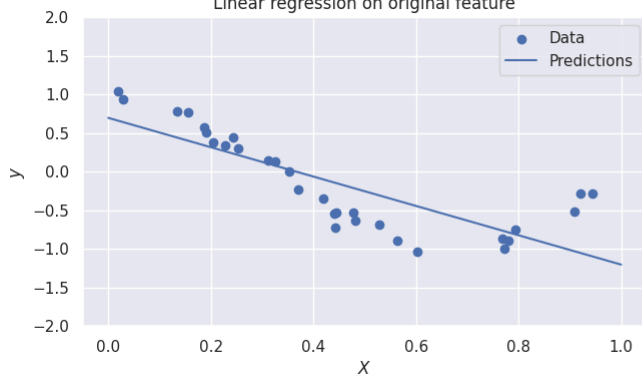
fig, axs = plt.subplots(figsize=(16, 4), ncols=2)

regr = LinearRegression()
regr.fit(X[:, np.newaxis], y)
y_pred_regr = regr.predict(x_plot[:, np.newaxis])
axs[0].scatter(X[:, np.newaxis], y, label="Data")
axs[0].plot(x_plot, y_pred_regr, label="Predictions")
axs[0].legend()
axs[0].set_title("Linear regression on original feature")
axs[0].set_xlabel("$X$")
axs[0].set_ylabel("$y$")
axs[0].set_ylim(-2, 2)

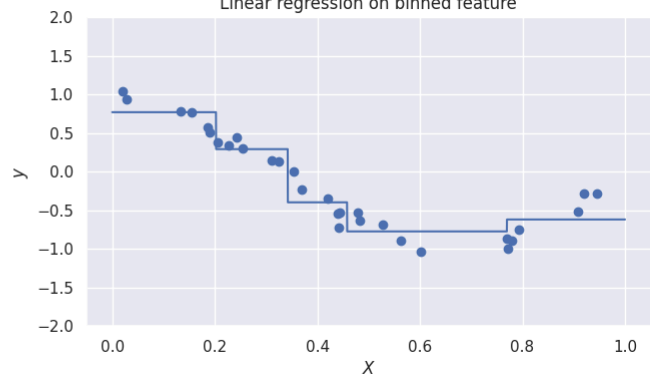
binner = KBinsDiscretizer(n_bins=5, strategy="quantile")
pipeline = Pipeline(steps=[("binning", binner), ("regression", LinearRegression())])
pipeline.fit(X[:, np.newaxis], y)
y_pred_binned = pipeline.predict(x_plot[:, np.newaxis])
axs[1].scatter(X[:, np.newaxis], y, label="Data")
axs[1].plot(x_plot, y_pred_binned, label="Predictions")
axs[1].set_title("Linear regression on binned feature")
axs[1].set_xlabel("$X$")
axs[1].set_ylabel("$y$")
axs[1].set_ylim(-2, 2)
```

Out[45]: (-2.0, 2.0)

Linear regression on original feature



Linear regression on binned feature



Видно, что качество модели существенно возросло. С другой стороны, увеличилось и количество параметров модели (из-за увеличения числа признаков), поэтому при бинаризации важно контролировать переобучение.

Иногда может помочь преобразование целевой переменной. Может оказаться, что по мере роста признаков целевая переменная меняется экспоненциально. Например, может оказаться, что при линейном уменьшении продолжительности видео число его просмотров растёт экспоненциально. Учесть это можно с помощью логарифмирования целевой переменной — ниже синтетический пример с такой ситуацией.

```
In [46]: X = np.random.exponential(1, size=30)
y = np.exp(X) + np.random.normal(scale=0.1, size=X.shape)
```

```
x_plot = np.linspace(np.min(X), np.max(X), 10000)
```

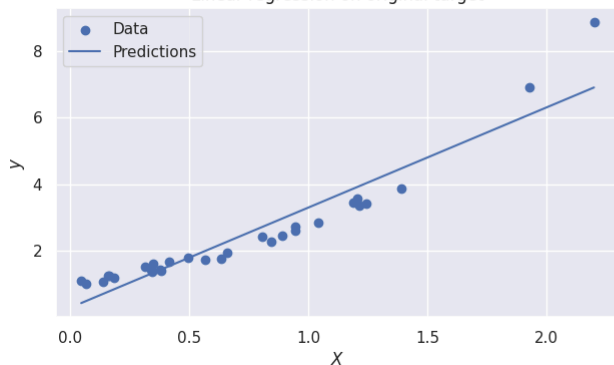
```
fig, axs = plt.subplots(figsize=(16, 4), ncols=2)
```

```
regr = LinearRegression()
regr.fit(X[:, np.newaxis], y)
y_pred_regr = regr.predict(x_plot[:, np.newaxis])
axs[0].scatter(X[:, np.newaxis], y, label="Data")
axs[0].plot(x_plot, y_pred_regr, label="Predictions")
axs[0].legend()
axs[0].set_title("Linear regression on original target")
axs[0].set_xlabel("$X$")
axs[0].set_ylabel("$y$")
```

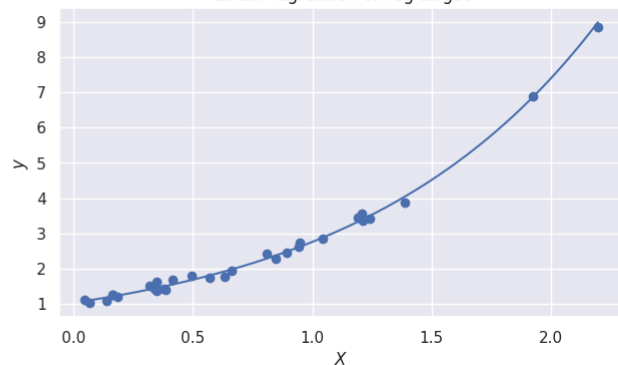
```
y_log = np.log(y)
regr.fit(X[:, np.newaxis], y_log)
y_pred_log = np.exp(regr.predict(x_plot[:, np.newaxis]))
axs[1].scatter(X[:, np.newaxis], y, label="Data")
axs[1].plot(x_plot, y_pred_log, label="Predictions")
axs[1].set_title("Linear regression on log target")
axs[1].set_xlabel("$X$")
axs[1].set_ylabel("$y$")
```

```
Out[46]: Text(0, 0.5, '$y$')
```

Linear regression on original target



Linear regression on log target



Но, конечно, вряд ли в реальных данных будет действительно экспоненциальная связь между целевой переменной и линейной комбинацией признаков. Тем не менее, логарифмирование всё равно может помочь.

Часть 5. Свободный полет

Оцените качество лучшей модели на тестовой выборке. Сравните с результатами на leaderboard, вы также можете отправить свое решение. Вы можете продолжить экспериментировать и постараться получить качество как можно выше.

```
In [47]: test_data = pd.read_csv("test.csv")
test_data.head()
```

```
Out[47]:
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	ScreenPorch	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType	SaleCondition
0	1461	20	RH	80.0	11622	Pave	NaN	Reg	Lvl	AllPub	...	120	0	NaN	MnPrv	NaN	0	6	2010	WD	Normal
1	1462	20	RL	81.0	14267	Pave	NaN	IR1	Lvl	AllPub	...	0	0	NaN	NaN	Gar2	12500	6	2010	WD	Normal
2	1463	60	RL	74.0	13830	Pave	NaN	IR1	Lvl	AllPub	...	0	0	NaN	MnPrv	NaN	0	3	2010	WD	Normal
3	1464	60	RL	78.0	9978	Pave	NaN	IR1	Lvl	AllPub	...	0	0	NaN	NaN	NaN	0	6	2010	WD	Normal
4	1465	120	RL	43.0	5005	Pave	NaN	IR1	HLS	AllPub	...	144	0	NaN	NaN	NaN	0	1	2010	WD	Normal

5 rows × 80 columns

```
In [48]: ids = test_data["Id"]
del test_data["Id"]
X = test_data

numeric_data = X.select_dtypes([np.number])
numeric_features = numeric_data.columns

for col in numeric_features:
    X_mean = X_train[col].mean()
    X[col].fillna(X_mean, inplace=True)
```

```
In [49]: categorical = list(X.dtypes[X.dtypes == "object"].index)
X[categorical].head()

for col in categorical:
    X[col].fillna("NotGiven", inplace=True)
```

```
In [50]: column_transformer = ColumnTransformer(  
        [ ("ohe", OneHotEncoder(handle_unknown="ignore"), categorical), ("scaling", StandardScaler(), numeric_features)]  
        )  
pipeline = Pipeline(steps=[("ohe_and_scaling", column_transformer), ("regression", Ridge())])  
  
model = pipeline.fit(X_train_1, y_train_1)  
y_pred = model.predict(X)  
  
y_pred
```

```
Out[50]: array([[123023.75178685, 123063.90846091, 191269.81264716, ...,  
        168848.79575395, 121494.48958926, 200957.43482471], shape=(1459,)])
```

```
In [52]: output = pd.DataFrame({"Id": ids, "SalePrice": y_pred})  
  
output.to_csv('predict.csv', index=False)
```